

LineageBench: a cited benchmark for weight-based model lineage

Muhammad Awais Bin Adil · Saad Aamir Reference implementation: [modelDNA](#) · Dataset: huggingface.co/datasets/AwaisAdilKhokhar/lineagebench

The question, and why the obvious ground truth is poisoned

Most open-weight models on the Hub do not carry trustworthy parentage. A large share have no documented base at all; others carry a self-reported `base_model` tag that no one has ever checked against the weights. That gap is where license disputes, merge archaeology, and safety-provenance questions all live, and the recurring question underneath them is the same: *which foundation model is this thing actually derived from?*

Answering it from weights alone is a measurable task, so it should have a benchmark. But a lineage benchmark has a peculiar problem: the label you would reach for first — the model's own `base_model` field — is exactly the thing a lineage tool exists to audit. Grading a weight-based method against self-reported tags would reward it for agreeing with claims that might be false, which is the opposite of what the method is for.

LineageBench takes its ground truth from somewhere the tool cannot see and the publisher cannot fudge after the fact: the **publishing organization's own documentation** — technical reports, official release notes, model cards written by the org that trained the *parent*. Every label ships with a citation. The benchmark is small and says so, but every row is sourced, and every number in it regenerates offline from a committed fingerprint cache in seconds.

What's in it

Fifteen suspect models across five foundation families, each judged against a pool of eight curated foundation parents. The suspects are chosen to cover the derivation regimes a real scanner actually meets:

kind	suspects	ground-truth source
fine-tune	Zephyr-7B- β , OpenHermes-2.5, Nous-Hermes, Dolphin-2.9	community model cards + tech reports
continued-pretrain	CodeLlama-7B, Qwen2.5-Coder-7B-Instruct	Meta / Qwen technical reports
official-instruct	Llama-3-8B-Instruct, Qwen2.5-{7B,14B}-Instruct, Mistral-7B-Instruct-v0.3, Gemma-2-9b-it, Gemma-2b-it	the orgs' own instruct releases
merge-chain	AlphaMonarch-7B	mlabonne model card (DARE-TIES over Mistral fine-tunes)
depth-upscale	SOLAR-10.7B-v1.0	SOLAR paper (arXiv:2312.15166) — <i>limitation case</i>
quantized-copy	Mistral-7B-v0.1-GPTQ	TheBloke GPTQ 4-bit repack — <i>limitation case</i>

Thirteen of the fifteen form the **positive pool**. The last two are marked out-of-scope on purpose (`in_positive_pool: false`) and scored separately, because for a shape-matched, safetensors-only comparator the *correct* answer on a 32→48-layer depth-upscale and on a packed-int4 requantization is **abstention**, not a confident match. A method that scored those as positives would be wrong. Keeping them in the benchmark — graded against their documented expected behavior — is how you check that a tool fails honestly.

The scoring protocol

The design goal is to make the benchmark hard in the way the real task is hard, and easy nowhere.

Attribution, not isolated scoring. Each suspect is run through the full verdict engine against *all eight* candidate parents. The true parent has to *win*, not merely score highly on its own. This is what a real scan faces: a model arrives with no reliable label and the tool has to pick the right base out of a field of plausible ones.

Hard negatives that a real scan would actually hit. The negative pool is 107 pairs: every positive suspect scored against each *cross-family* wrong parent, plus all cross-family parent-versus-parent comparisons. These are the false-positive surface — the pairings where a naive similarity metric might fire on shared architecture or shared pretraining-data statistics.

Same-family wrong parents are deliberately excluded from the negatives. Scoring Zephyr (a Mistral-v0.1 fine-tune) against Mistral-v0.3 as a *negative* would punish the tool for detecting something that is genuinely true — those two bases share lineage. A benchmark that counted real relatedness as a false alarm would be measuring the wrong thing. So the negative pool is cross-family only, and the positive labels stay honest about which base is the true one.

Results (modelDNA v0.1)

metric	value
AUROC (13 positives vs 107 hard negatives)	1.0
TPR at 1% FPR	1.0
false positives at the $p \geq 0.9$ reporting threshold	0 / 107
weakest positive	0.9426 (gemma-2b-it)
strongest negative	0.6596
top-1 parent attribution	13 / 13

The separation is clean: the weakest true positive sits at 0.94, the strongest false alarm at 0.66, and the entire abstention band from 0.50 to 0.90 is empty on this data. Broken out by derivation kind, the weakest positive is 0.998 for fine-tunes, 0.9984 for the merge chain, 0.9796 for continued-pretrains, and 0.9426 for official instructs — the heavily-tuned official instructs are the hardest positives, as you would expect, and they still clear the threshold comfortably.

Both limitation cases behaved exactly as documented. SOLAR-10.7B produced an honest `NO_MATCH` with its layer-count mismatch reported rather than papered over; the tool does not yet attempt a depth-alignment search, and it says so instead of guessing. The GPTQ copy abstained on the packed int4 tensors while still ranking its true parent first on the signals it could compute.

Two caveats we would rather report than have you discover

An AUROC of 1.0 is an invitation to look for what the top-line metric hides. Two things.

AlphaMonarch-7B resolved to the right family but as `SAME_FAMILY_UNRESOLVED`, not `LIKELY_MERGE`.

It is a DARE-TIES merge, but every one of its ancestors is a Mistral fine-tune, so there is no cross-family split for the scan-level merge heuristic (which is family-based in v0.1) to notice. The attribution is correct — it traces to the right Mistral root — but the *merge* is not flagged at scan time. modelDNA's separate `decompose` tool, given a candidate list, resolves this model completely; the scan alone does not.

Nous-Hermes was attributed correctly but classed `QUANTIZED_COPY` rather than `FINE_TUNE`. Its fine-tune moved the weights so little that its parameter-sample cosine against Llama-2-7b (0.9998) crosses the dtype-recast rule's 0.999 threshold, and because the two repos ship in different dtypes (bfloat16 vs the parent's float16) the recast rule fires before the fine-tune rule. The parent, the probability, and the ranking are all correct, so the headline metrics are unaffected — but the boundary between “recast of the same values” and “very light fine-tune” needs a delta-magnitude discriminator, not a cosine threshold.

Neither changes the ranking or the AUROC. Both are the kind of thing a benchmark exists to surface.

Scope, stated plainly

This is a deliberately small slice — 15 pairs, not the ≥ 300 the project's planning document sets as the eventual target. Read the AUROC of 1.0 as “*no errors at this scale*”, not as a claim that errors are impossible; 13 positives cannot validate a calibration curve, and the perfect ranking is more

trustworthy than the exact probability magnitudes, which run conservative on the most heavily-tuned derivatives.

Growing it is mechanical. The expensive part of adding a pair is fingerprinting the model, and that pipeline is fast and automated; the scarce input is a *cited, org-documented* label. Contributions of new pairs — with a citation to the publishing org’s own documentation, never to a `base_model` tag — are welcome via pull request to the [manifest](#).

And the standing caveat of every weight-space method: absence of a detected edge is not evidence of independence. Distillation leaves no weight linkage and is invisible to this and every similar method by construction; depth and quantization mismatches abstain rather than resolve. A verdict is *statistical consistency with derivation against a measured background*, never an accusation.

Reproduce it

Every label carries a citation and every fingerprint is cached, so the whole benchmark regenerates offline — no Hub round-trip, no gated licenses:

```
pip install modeldna
git clone https://github.com/AwaisAdilKhokhar/modelDNA
python modelDNA/benchmarks/real_lineagebench.py --no-fetch
# -> ship gates: PASS
```

The named dataset — ground truth, the reference implementation’s per-pair results, the candidate-parent list, and the headline metrics — loads directly:

```
from datasets import load_dataset

gt = load_dataset("AwaisAdilKhokhar/lineagebench", "ground_truth", split="train")
res = load_dataset("AwaisAdilKhokhar/lineagebench", "reference_results", split="train")
```

Artifacts

- **Dataset:** <https://huggingface.co/datasets/AwaisAdilKhokhar/lineagebench>
- **Reference implementation:** <https://github.com/AwaisAdilKhokhar/modelDNA>
(`benchmarks/real_lineagebench.py`, Apache-2.0)
- **Ground-truth manifest with per-pair citations:** [benchmarks/lineagebench_pairs.json](#)
- **Method write-up** (the four-signal fingerprint and the calibrated verdict engine behind the numbers): [modelDNA tech report](#)

Cite

```
@misc{lineagebench,  
  title = {LineageBench: org-documented lineage ground truth for open-weight LLMs},  
  author = {Awais Bin Adil, Muhammad and Aamir, Saad},  
  year = {2026},  
  howpublished = {\url{https://huggingface.co/datasets/AwaisAdilKhokhar/lineagebench}},  
  note = {Reference implementation: modelDNA, \url{https://github.com/AwaisAdilKhokhar/modelDNA}}  
}
```